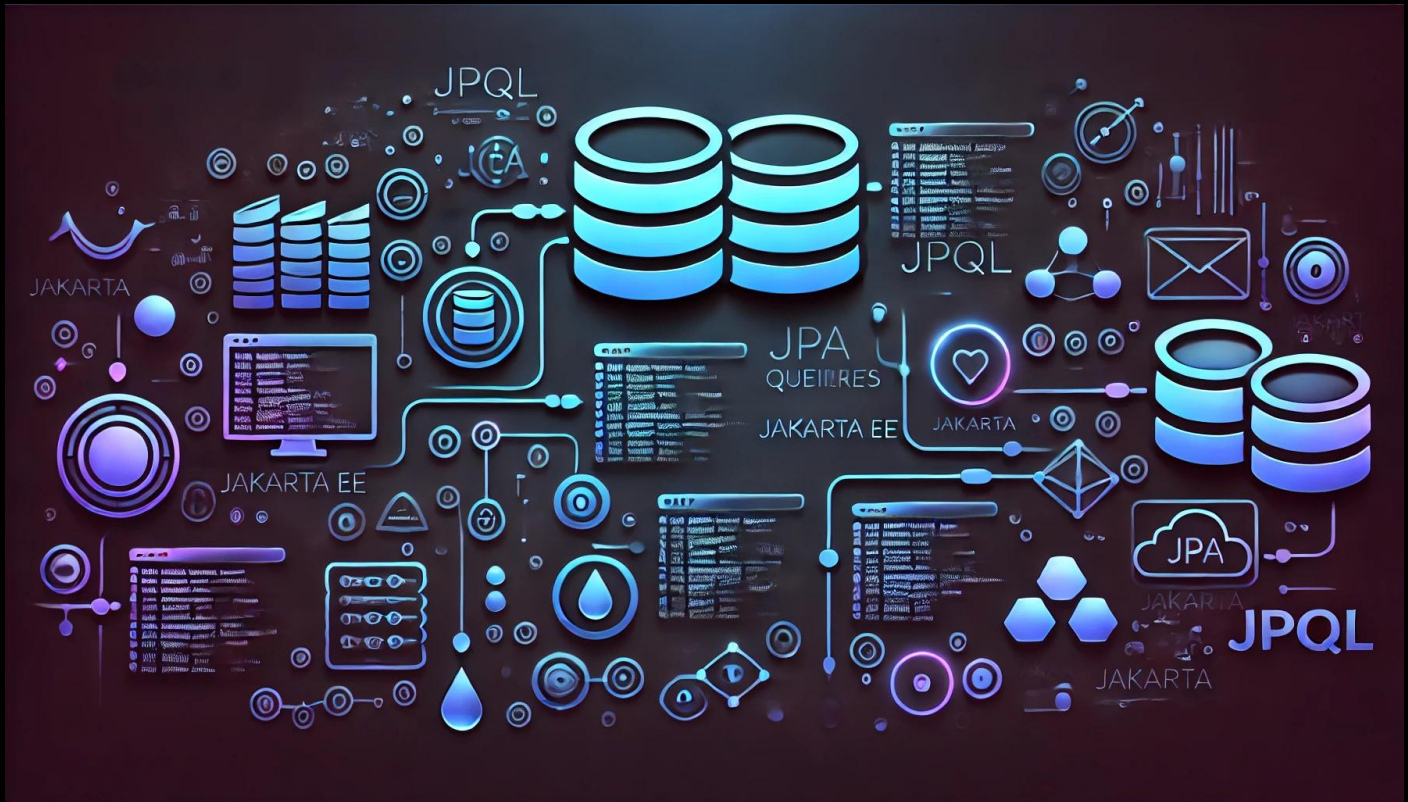


JPQL con JPA y Jakarta EE – parte 6



■ Consultas con JPQL en JPA - Contar Nombres Distintos y Concatenación de Campos

◆ Introducción

En esta guía, aprenderemos a utilizar **JPQL en JPA** para realizar dos consultas avanzadas:

1. **Contar los nombres únicos** en la base de datos usando `COUNT (DISTINCT)`.
2. **Concatenar y convertir a mayúsculas** el nombre y apellido de las personas con `CONCAT ()`.

Estas consultas nos permiten manejar y analizar información de manera más eficiente sin necesidad de procesar los datos después de obtenerlos.

✦ Ejemplo práctico:

- Queremos contar **cuántos nombres distintos existen** en la base de datos.
- Queremos obtener el **nombre completo** de cada persona concatenando su nombre y apellido.

1 Conceptos Básicos

📌 ¿Cómo contar valores distintos en JPQL?

JPQL nos permite contar valores distintos con `COUNT(DISTINCT atributo)`.

Ejemplo en SQL:

```
SELECT COUNT(DISTINCT nombre) FROM persona;
```

Equivalente en JPQL:

```
SELECT COUNT(DISTINCT p.nombre) FROM Persona p;
```

📌 ¿Cómo concatenar campos en JPQL?

JPQL permite concatenar dos o más valores con `CONCAT(valor1, valor2)`.

Ejemplo en SQL:

```
SELECT CONCAT(nombre, ' ', apellido) FROM persona;
```

Equivalente en JPQL:

```
SELECT CONCAT(p.nombre, ' ', p.apellido) FROM Persona p;
```

2 Solución Paso a Paso

🚀 2.1 Actualizar la Clase `PruebaJPQL`

La clase `PruebaJPQL` ejecutará las consultas y mostrará los resultados.

```
package sga.cliente.jpql;

import java.util.List;
import jakarta.persistence.*;
import java.util.Iterator;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
public class PruebaJPQL {

    private static final Logger log = LoggerFactory.getLogger(PruebaJPQL.class);

    public static void main(String[] args) {
        String jpql = null;
        List<Persona> personas = null;

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {

            //1. Consulta de todos los objetos de tipo Persona
            log.info("\n1. Consulta de todas las Personas");
            jpql = "select p from Persona p";
            personas = em.createQuery(jpql, Persona.class).getResultList();
            mostrarPersonas(personas);

            //2. Consulta de una persona por ID
            log.info("\n2. Consulta de Persona con ID = 1");
            jpql = "SELECT p FROM Persona p WHERE p.idPersona = 1";
            Persona persona = em.createQuery(jpql, Persona.class).getSingleResult();
            log.info("Persona encontrada: " + persona);

            //3. Consulta de una persona por nombre
            log.info("\n3. Consulta de Persona por nombre 'Ivonne'");
            jpql = "SELECT p FROM Persona p WHERE p.nombre = 'Ivonne'";
            personas = em.createQuery(jpql, Persona.class).getResultList();
            mostrarPersonas(personas);

            //4. Consulta de datos individuales (tupla)
            log.info("\n4. Consulta de datos individuales: nombre, apellido, email");

            jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
            Iterator iter = em.createQuery(jpql).getResultList().iterator();

            while (iter.hasNext()) {
                Object[] tupla = (Object[]) iter.next();
                String nombre = (String) tupla[0];
                String apellido = (String) tupla[1];
                String email = (String) tupla[2];

                log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
            }
        }
    }
}
```

```
//5. Obtiene el objeto Persona y el id, se crea un arreglo de tipo Object con 2 columnas
log.info("\n5. Obtiene el id y el objeto Persona, se crea un arreglo de tipo Object con 2
columnas");
Object[] tupla = null;
jpql = "select p.idPersona, p from Persona p ";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
    int idPersona = (int) tupla[0];
    persona = (Persona) tupla[1];
    log.info("ID persona:" + idPersona);
    log.info("Objeto persona:" + persona);
}

//6. Consulta de todas las personas
log.info("\n6. Consulta de los ids de todas las personas");
jpql = "SELECT p.idPersona FROM Persona p";
List<Integer> ids = em.createQuery(jpql, Integer.class).getResultList();
ids.forEach(idPersona -> log.info("ID persona: " + idPersona));

//7. Regresa el valor minimo y maximo del idPersona (scaler result)
log.info("\n7. Regresa el valor minimo y maximo del idPersona (scaler result)");
jpql = "select MIN(p.idPersona) as MinId, MAX(p.idPersona) as MaxId, COUNT(p.idPersona) as
Contador from Persona p";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
    Integer idMin = (Integer) tupla[0];
    Integer idMax = (Integer) tupla[1];
    Long count = (Long) tupla[2];
    log.info("idMin:" + idMin + ", idMax:" + idMax + ", count:" + count);
}

//8. Cuenta los nombres de las personas que son distintos
log.info("\n8. Cuenta los nombres de las personas que son distintos");
jpql = "select COUNT(distinct p.nombre) from Persona p";
Long contador = (Long) em.createQuery(jpql).getSingleResult();
log.info("no. de personas con nombre distinto:" + contador);

//9. Concatena y convierte a mayusculas el nombre y apellido
log.info("\n9. Concatena y convierte a mayusculas el nombre y apellido");
jpql = "select CONCAT(p.nombre, ' ', p.apellido) as Nombre from Persona p";
List<String> nombres = em.createQuery(jpql).getResultList();
for (String nombreCompleto : nombres) {
    log.info(nombreCompleto);
}
```

```
    }  
}  
}  
  
private static void mostrarPersonas(List<Persona> personas) {  
    for (Persona p : personas) {  
        log.info("Persona: " + p);  
    }  
}  
}
```

3 Contar los Nombres Distintos

✦ Queremos contar cuántos nombres únicos existen en la base de datos.

```
String jpql = "SELECT COUNT(DISTINCT p.nombre) FROM Persona p";  
Long contador = (Long) em.createQuery(jpql).getSingleResult();  
log.info("Número de nombres distintos: " + contador);
```

🔗 Explicación:

- `COUNT(DISTINCT p.nombre)`: Cuenta cuántos nombres diferentes existen en la base de datos.
- `createQuery(jpql).getSingleResult()`: Ejecuta la consulta y devuelve un único resultado.

4 Concatenar y Convertir a Mayúsculas el Nombre y Apellido

✦ Queremos obtener el nombre completo de cada persona concatenando nombre y apellido.

```
String jpql = "SELECT CONCAT(p.nombre, ' ', p.apellido) FROM Persona p";  
List<String> nombres = em.createQuery(jpql, String.class).getResultList();  
nombres.forEach(nombreCompleto -> log.info("Nombre Completo: " + nombreCompleto));
```

🔗 Explicación:

- `CONCAT(p.nombre, ' ', p.apellido)`: Une el nombre y apellido, separándolos con un espacio.
- `createQuery(jpql, String.class).getResultList()`: Devuelve una lista de nombres completos en formato `String`.

5 Resultado Esperado

Si ejecutamos la clase `PruebaJPQL`, veremos en la consola:

- 8. Cuenta los nombres de las personas que son distintos
- [EL Fine]: sql: ServerSession(2133344792)--Connection(553672556)--SELECT COUNT (DISTINCT(NOMBRE)) FROM PERSONA
- [main] INFO sga.cliente.jpql.PruebaJPQL - no. de personas con nombre distinto:4
- [main] INFO sga.cliente.jpql.PruebaJPQL -
- 9. Concatena y convierte a mayusculas el nombre y apellido
- [EL Fine]: sql: ServerSession(2133344792)--Connection(553672556)--SELECT CONCAT (CONCAT (NOMBRE, ?), APELLIDO) FROM PERSONA
- bind => []
- [main] INFO sga.cliente.jpql.PruebaJPQL - Rafael Juarez
- [main] INFO sga.cliente.jpql.PruebaJPQL - Ivonne Lara
- [main] INFO sga.cliente.jpql.PruebaJPQL - Maria Esparza
- [main] INFO sga.cliente.jpql.PruebaJPQL - Hugo Hernandez

Esto confirma que **las consultas JPQL están funcionando correctamente** 🎯.

6 Conclusión

🎯 En esta guía aprendimos a utilizar **JPQL en JPA** para contar nombres distintos y concatenar atributos de entidad.

♦ Puntos clave:

- ✓ `COUNT (DISTINCT)` permite contar valores únicos en JPQL.
- ✓ `CONCAT ()` permite unir valores en una sola cadena de texto.
- ✓ **JPQL facilita el procesamiento de datos dentro de la consulta sin manipulación adicional en Java.**

🚀 ¡Ahora puedes aplicar estas técnicas en tus proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)